Team #88 — M1: Approach

Problem Statement 1: AI Incident Report Assistant (Qualcomm QIDK)

1. Problem statement and our understanding

Official Problem Statement (Verbatim from Qualcomm Challenge):

> "**Problem Statement 1 – AI Incident Report Assistant:** End-to-end voice-driven incident reporting using on-device Generative AI and function calling. **On-Device AI Challenge:** Voice in. Structured report out.
> **Challenge & Workflow:** Make incident reporting as easy as speaking. Natural conversation · Accurate extraction · On-device execution: SPEAK → UNDERSTAND → EXTRACT → REVIEW → SUBMIT.
> **Example Fields of a Form:** The application should automatically populate fields such as: Incident Type, Severity Level, Incident Location, Date & Time, Persons Involved, Equipment / Asset Involved, Incident Description, Immediate Actions Taken, Witness Information, Recommended Follow-Up Action.
> **Example Input:** *"There was a water leak near Conference Room 2 around 10 AM. Maintenance has already isolated the area."*
> **Example Output:** Incident Type → Water Leak; Location → Conference Room 2; Time → 10 AM; Action Taken → Area Isolated.
> **Evaluation Criteria:** (1) All AI models must execute on-device using Snapdragon acceleration. Strictly no cloud APIs. (2) Ability to understand natural conversational speech rather than predefined commands. (3) Accuracy of speech recognition and transcription. (4) Accuracy of extracting incident information from user narration. (5) Correct mapping of extracted information to report fields."

Our Understanding and Technical Strategy: Manual incident forms are slow and inconsistent, especially under stress. Turning free-form spoken accounts into complete, verified reports offline requires addressing key real-world challenges:

- **Strict On-Device Acceleration:** No cloud fallback. Operates fully in airplane mode on Snapdragon 8 Elite (`SM8750P`), Hexagon **V79** NPU, QAIRT 2.45, Genie 1.17 (`QnnHtp`).
- **Natural Speech Disfluency:** Spoken input has non-chronological order, fillers, self-corrections, and code-mixing (Hinglish, Telugu); parsing cannot rely on fixed phrasing.
- **Deterministic Guardrails (Grounding):** Small edge LLMs risk hallucination. Under our principle **"the model proposes, the code verifies"**, typed tool calls propose candidate values while Kotlin deterministic rules enforce strict grounding against the transcript.
- **Active Conversational Clarification:** When mandatory fields are absent, the assistant engages in targeted spoken follow-up rather than accepting incomplete submissions.

2. Idea / abstract

We build an **offline Android assistant** that transcribes spoken accounts on the Hexagon NPU, extracts report entities via typed **function calling**, and verifies every field against transcript evidence using deterministic Kotlin rules. Severity is assigned via a **5-tier rulebook** distilled from OSHA, WHO, and KABCO safety taxonomies. Missing mandatory fields trigger targeted spoken follow-up queries. Planned extensions:

- **Multilingual input:** Hindi, Telugu, and Hinglish speech-in; report fields normalized to English.
- **Photo evidence:** On-device CLIP hazard classifier suggests incident type and severity from scene photos.
- **Emergency escalation:** Critical incidents trigger a 10s cancellable countdown alerting emergency contacts.
- **Offline spatial intelligence:** Severity-coded incident map and history-aware preventive tips for recurring issues.

3. High-level approach, pipeline and architecture

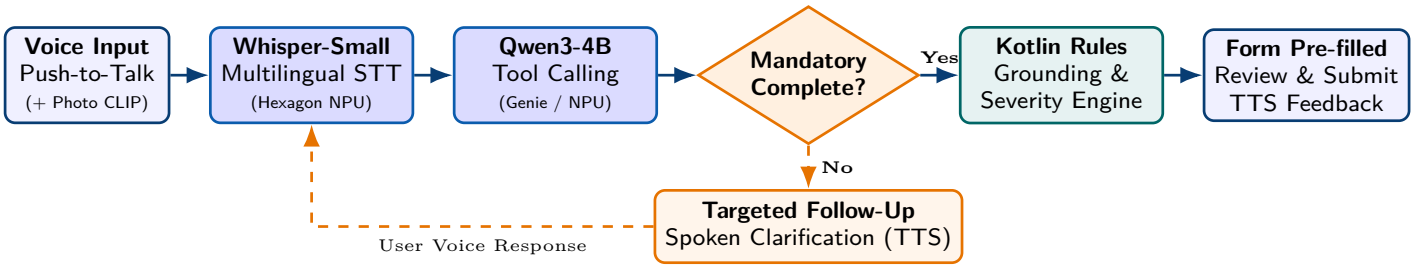


Figure 1: End-to-end architecture: NPU acceleration handles speech and reasoning; Kotlin enforces grounding and follow-up.

User flow / UI. Dark-mode app with a primary **Mic button** and optional **Camera button**. States: `IDLE` → `RECORDING` → `PROCESSING` (STT → LLM → JSON → field check) → `CLARIFICATION` (targeted voice follow-up loop) or `COMPLETED` (form filled, TTS: *"Done, incident reported"*).

Stage	Model / component (all on-device)	Runtime / accelerator
Voice input	Push-to-talk + energy-threshold VAD	CPU
Speech-to-text	Whisper-Small multilingual (fallback Whisper-Base). English transcribe, Hindi/Telugu translate to English	QNN, Hexagon NPU
Photo cues (optional)	OpenAI CLIP zero-shot hazard labels ("water leak", "fire", "wet floor")	QNN/TFLite, Hexagon NPU
Extraction + tools	**Qwen3-4B-Instruct-2507, w4a16** (~3.2 GB, ~27 tok/s). Fallback: Llama 3.2 1B (w4a16)	Genie / QAIRT 2.45, Hexagon NPU
Rule & safety layer	`IncidentRules.kt`: tolerant JSON parser, grounding, severity rules, `FormManager` follow-up loop	CPU (pure Kotlin)
Speech output	QIDK sample TTS (fallback: Android offline `TextToSpeech`)	On-device
App shell	Qualcomm QIDK **ASR-LLM-TTS** Android sample (Snapdragon 8 Elite / V79 / Android 15)	Android

Tool calling and schema. Small typed tools (≤3 parameters each): `recordWhat`, `recordWhereWhen`, `recordPeople`, `recordResponse`, `findSimilarIncidents`, `raiseEmergency`. They populate a structured report (`incident_type` ∈ {Water Leak, Fire, Injury, Accident, Equipment Failure, Electrical Hazard, Gas Leak, Spill, Security, Other}, severity, location, datetime, persons, equipment, actions taken, witnesses, follow-up), validated and stored locally. Prompting uses few-shot chat turns, answer pre-filling (`{"incident_type":`), and deterministic decoding (`temp 0.1`, `top-k 1`).

Core Design Principles:

- **LLM handles language; code handles rules:** Mandatory-field checks, normalisation, and datetime offsets are deterministic.
- **Strict transcript grounding:** Unmentioned fields stay empty; extracted entities must match transcript spans (fuzzy snap-back corrects minor ASR typos, e.g. "catering stairs" → "cafeteria stairs"). Code synthesizes final description from grounded facts.
- **Template follow-ups:** Mandatory fields (type, location, date/time, severity) asked at most twice, then marked "Not provided".
- **Safety net:** Lexical triggers (Hindi *aag, chot, khoon*; Telugu *nippu, debba, neellu*) prevent critical events from being dropped.

Severity estimation. Narratives are evaluated across impact, intervention level, duration, and mitigations into 5 discrete tiers:

Level (form value)	Criteria	Indicative evidence
L0 None / Near-miss	No harm, or hazard safely neutralized in time	"near-miss", "caught in time", "no injury"
L1 Low	Mild, transient impact (hours); minimal or no first aid	"first aid", "minor pain", "resolved in hours"
L2 Medium	Moderate impact (days); clinic/ED evaluation, localised hazard	"ED visit", "stitches", "isolated", "degradation"
L3 High	Severe harm, major fire/gas release, ICU, surgery, critical outage	"amputation", "ICU", "surgery", "collapse"
L4 Critical	Fatalities, mass casualties, trapped persons, catastrophic loss	"fatal", "death", "ambulance", "mass casualty"

Decision rules: Irreversible outcomes prioritized (fatality → L4, surgery → L3); hazards (fire, gas) floor at L2; negations suppress false alarms ("no one hurt"); ambiguous cases conservatively assign the higher severity level.

4. Planned testing methodology

Test sets. (a) **35-case typed development set** across 7 categories: clean (11), noisy/fillers (4), missing info (5), non-incident (3), Hinglish (6), ASR noise (3), negation (3); plus 7 Romanised-Telugu cases. (b) **20–30 spoken on-device recordings** with reference transcripts. (c) **Held-out evaluation set** authored independently. (d) ∼20 incident scene photos.

Evaluation Dimension	Target Metric
Speech recognition	Word Error Rate (WER) vs. reference transcripts; transcribe vs. translate quality on Indic audio
Extraction & mapping	Per-field accuracy, precision/recall/F1, exact match; valid-JSON rate; hallucination rate
Mandatory-field handling	% missing fields resolved via voice follow-up; average follow-up conversational turns
Tool-calling reliability	% valid tool invocations with conforming schema and typed arguments
Latency & acceleration	Stage latencies (ASR, LLM time-to-first-token, tok/s, end-to-end); Hexagon NPU vs. CPU profiling
Offline integrity	Complete execution verified in airplane mode with zero external network socket calls

Evaluation harness & early findings. Automated harness (`eval.ps1`) profiles CLI and APK runs on QIDK, logging per-field CSVs. A suite of **24 Kotlin checks** (`IncidentRulesCheck.kt`) tests edge outputs. Early validation results on dev set:

- **Llama 3.2 1B (w4a16):** 68.6% valid JSON, 47.9% field accuracy, 8 hallucinated fields, 1.77 s latency.
- **Qwen3-4B-Instruct (w4a16):** 100% valid JSON, 92.1% accuracy, 2 hallucinated fields, 5.24 s latency.
- **Qwen3-4B + Kotlin Rule Layer: 100%** valid JSON, **100%** accuracy, **0** hallucinated fields; 5/6 follow-ups resolved; 24/24 unit checks pass.